DATA DEFINITION LANGUAGE

Data Definition Language: This is a 1st sub Language in SQL which is used to define the database objects such as table, view etc.

- This language contains five commands
  1. Create
  2. Alter
  3. SP_Rename
  4. Truncate
  5. Drop

1. Create:

- This command is used to create the database objects within the database

Syntax: CREATE TABLE <TABLE NAME>
(COL 1 DATA TYPE (size),
COL2 DATA TYPE (size),
:
:
:
COLN DATA TYPE (size));

Ex: CREATE TABLE EMP (EID Int, ENAME Varchar (15), SAL DECIMAL (6, 2));

Rules for Creating a Table:

- Table name must be unique under the database.
- Column names must be unique within the table.
- Never start table name with numeric or special characters except underscore"_".
- Do not use space in table name if we want give space in table name then use underscore symbol only.
- Every object name should contain minimum one character and maximum 128 characters.
- The maximum no. of columns a table can have 1024 columns.

2. ALTER:

- This command is used to modify the structure of a table using this command, we can perform four different operations
    - Using this command we can increase (or) decrease the size of the data type & also we can change the data type from old data type to new data type
    - We can add a new column to the existing table
    - We can change the column name from old column name to new column name
    - We can remove the column from the existing table
- This command contains 4 sub commands
    1. ALTER- ALTER COLUMN
    2. ALTER- ADD
    3. SP_RENAME
    4. ALTER- DROP

a. ALTER-ALTER COLUMN:

- **Syntax:** ALTER TABLE <TABLE NAME> ALTER COLUMN <COLUMN NAME> DATA TYPE (SIZE)
- **Ex:** ALTER TABLE EMP ALTER COLUMN ENAME char (25);

b. ALTER-ADD:

- **Syntax:** ALTER TABLE <TABLE NAME> ADD <COLUMNNAME >DATA TYPE(size);
- **Ex:** ALTER TABLE EMP ADD DEPTNO int;

c. ALTER-DROP:

- **Syntax:** ALTER TABLE <TABLE NAME> DROP COLUMN <COLUMN NAME>;
- **Ex:** ALTER TABLE EMP DROP COLUMN SAL;

d. SP_RENAME:

- **Syntax:** SP_RENAME „TABLENAME.OLDCOLUMN",'NEW COLUMN NAME‟,‟COLUMN,;
- **Ex:** SP_RENAME „EMP.SAL",‟SALARY",'COLUMN"

3. SP_RENAME:

- This command is used to change the table name from old table name to new table name
- **Syntax:** SP_Rename „old table name",' New table name"
- **Ex:** SP_Rename „EMP‟,'EMP1"

4. TRUNCATE:

- This command is used for to delete all the records from existing table permanently
- **Syntax:** TRUNCATE TABLE <TABLE NAME>
- **Ex:** TRUNCATE TABLE EMP;

5. DROP:

- This command is used to remove the table permanently from the database
- **Syntax:** DROP TABLE <TABLE NAME>
- **Ex:** DROP TABLE EMP;

Note: SP_help: This command is used to see the structure of table

- **Syntax:** SP_help <table name>
- **Ex:** SP_help EMP

Note: Syntax to view tables in the current database.

- **select * from sysobjects where XTYPE='u'**

DATA MANIPULATING LANGUAGE

Data Manipulating Language: This is the 2nd sub language in SQL, which is used to manipulate the data within database. This Language contains 4 commands

1. Insert
2. Update
3. Delete
4. Select

1. INSERT:

- Using this command we can Insert the records into the existing table
- We can insert the records into the table in two methods
  Explicit method
  Implicit method

Explicit method:

- In this method user has to enter all the values into all the columns without anything omitting (or) left any column data
- **Syntax:** INSERT INTO <TABLE NAME> VALUES <VAL1, VAL2, ….VALN>;

  (OR)

  INSERT <TABLE NAME> VALUES <VAL1, VAL2, .VALN>;
  (Here "INTO" Keyword is optional)

- **Ex1:** INSERT INTO EMP VALUES (101,"RAJ",9500);
- **Ex2:** INSERT EMP VALUES (101,"RAJ",9500); 1
  Row(s) affected

Implicit method:

- In this method we can enter the values into the required columns in the table, so that user can omit (or) left some columns data while he enters the records into the table
- If the user omit any column data in the table then it automatically takes NULL
- **Syntax:** INSERT INTO <TABLE NAME> (COL1, COL2….COLN) VALUES (VAL1, VAL2… VALN);
- **Ex:** INSERT INTO EMP (EID, SAL) VALUES (106,9999);

2. UPDATE:

- This command is used to modify the data in the existing table
- By using this command we can modify all the records in the table & also specific records in the table (Using „where" clause)
- **Syntax:** UPDATE <TABLE NAME> SET COL=VALUE;
- **Ex:** UPDATE EMP SET SAL=10000;

Syntax change for more than one data simultaneously

- **Syntax:** UPDATE <TABLE NAME> SET COL1=VALUE, COL2=VALUE……COLN=VALUE;
- **Ex:** UPDATE EMP SET EID=007,SAL=10000;

3. DELETE:

- This command is used to delete the records from existing table
- Using this command we can delete all the records and also to delete specific record (by using „where" clause)
- **Syntax:** DELETE FROM <TABLE NAME>
- **Ex:** DELETE FROM EMP;

  10 row(s) affected

Difference between TRUNCATE and DELETE Command:

SRNO	TRUNCATE	DELETE
01	It is a DDL command	It is a DML command
02	It is a permanent deletion	It is temporary deletion
03	Specific record deletion is not possible	We can delete the specific record
04	It doesn't support WHERE clause	It supports WHERE clause
05	We cannot Rollback the data	We can Rollback the data
06	Truncate will reset the identity Values	Delete will not reset the identity value

4. SELECT:

- This command is used to retrieve the data from existing table.
- Using this command we can retrieve all the records & also specific records from existing table (by using „where" clause)
- Using this command we can retrieve the data from the table in 3 ways
    1. **Projection**
    2. **Selection**
    3. **Joins**
- **Syntax:** SELECT * FROM <TABLE NAME>
- **Ex:** SELECT * FROM EMP;
- * represents all columns

Projection:

- Retrieving the data from specific columns is known as Projection
- **Syntax:** SELECT COL1,COL2……..COLN FROM <TABLE NAME>
- **Ex:** SELECT EID,ENAME FROM EMP;

Selection:

- Retrieving the data based on some condition is called selection
- In SQL, whenever we need to check a condition, we need to use a special clause called „where"
- **Syntax:** SELECT * FROM <TABLENAME> WHERE (CONDITION);
- **Ex:** SELECT * FROM EMP WHERE EID=101;

WHERE CLAUSE:

- This clause is used to check the condition based on the condition, we can retrieve, update, delete specific records in the table
- So we can apply the where clause only in select, update & delete

Select Command With Where clause:

- **Syntax:** SELECT * FROM <TABLE NAME> WHERE <CONDITION>
- **Ex:** SELECT * FROM EMP WHERE EID=102;

Update Command With Where clause:

- **Syntax:** UPDATE <TABLE NAME> SET <COLUMN NAME>=VALUE WHERE (CONDITION);
- **Ex:** UPDATE EMP SET ENAME="sai" WHERE EID=102;

Delete Command With Where clause:

- **Syntax:** DELETE FROM <TABLE NAME>WHERE <CONDITION>
- **Ex:** DELETE FROM EMP WHERE EID=102;

ALIAS:

- ALIAS is a duplicate name (or) alternate name for the original column name (or) Table name (or) an expression name.

- **Column level Alias:**
- **Syntax:** SELECT COLUMN NAME AS "ALIAS NAME",
  COLUMN NAME AS "ALIAS NAME",
  :
  :
  COLUMN NAME AS "ALIAS NAME" FROM <TABLE NAME>;
- **EX:** SELECT EID AS "EMPLOYEE ID", ENAME AS "EMPLOYEE NAME", SAL AS "SALARY" FROM EMP;
- **NOTE:** In the above example the keyword „as" is optional
- **EX:** SELECT EID "EMPLOYEE ID", ENAME "EMPLOYEE NAME", SAL "SALARY" FROM EMP;
- **NOTE:** In the above example quotations is also optional but there should not be space between column name
- **EX:** SELECT EID EMPLOYEEID, ENAME EMPLOYEENAME, SAL SALARY FROM EMP;
- **Ex:** SELECT EID EMPLOYEEID, ENAME EMPLOYEENAME, SAL SALARY, SAL*12 ANNUALSALARY FROM EMP;
- **EX:** SELECT EID EMPLOYEEID, ENAME EMPLOYEENAME, SAL SALARY FROM EMP WHERE ANNUALSALARY > 115000
- In the above example returns the runtime error message invalid column name „annual salary" because we cannot check the conditions on Alias name

IDENTITY: It is use to generate unique values in sequential order without user interaction. The default value of identity is Identity (1, 1).

Syntax: Identity (seed, increment)

Ex: CREATE TABLE EMP (EID INT IDENTITY (100, 1), ENAME VARCHAR (50));

Built In Functions(System Functions) IN SQL: SQL server provide number of built in functions like mathematical functions, character functions, date and time functions, aggregative functions,convertion functions etc.these can be used to perform certain operations and return a value.

Syntax: SELECT <Function Name> [Expressions]

Mathematical Functions: These functions perform a calculation based on input values provided as arguments, and return a numeric value.

ABS (): Returns the absolute, positive value of the given numeric expression.

Ex: select ABS(-15) ----15
select ABS(45) -----45

CEILING (): Returns the smallest integer greater than, or equal to, the given numeric expression.

Ex: select ceiling(15.000)----15
select ceiling(15.0001) --- 16
select ceiling(-12.34) ---- (-12)

FLOOR (): Returns the largest integer less than or equal to the given numeric expression.

Ex: select floor(15.000) -- 15
select floor(15.0001) ---15
select floor(-12.34) ---(-13)

SQUARE (): Returns the square of the given expression.

Ex: select SQUARE(5) --25

SQRT (): Returns the square root of the given expression.

Ex: select SQUARE(25) -- 5

POWER (n, m): Returns the power value of given expression

Ex: select POWER (2, 3) ----------8

SIGN (): Returns the positive (+1), zero (0), or negative (-1) sign of the given expression.

Ex: select SIGN(42)---------- 1
select SIGN(0) ----------0
select SIGN(-42)---------- (-1)

PI (): Returns the constant value of PI.

Ex: select PI() --------3.14159265358979
LOG (): Returns the natural logarithm of the given expression.

Ex: select LOG(2) ------0.693147180559945
LOG 10(): Returns the base-10 logarithm of the given expression.

Ex: select LOG10(10)--- 1

SIN (): Returns the trigonometric sine of the given angle (in radians) in an approximate numeric expression.

Ex: select SIN (0)-------0

COS (): A mathematic function that returns the trigonometric cosine of the given angle (in radians) in the given expression.

Ex: select COS (0)-------1

TAN (): Returns the tangent of the input expression.

Ex: select TAN (0)---------0

String Functions: These functions perform an operation on a string input value and return a string or numeric value.

ASCII (): Returns the ASCII code value of the leftmost character of a character expression.

Ex: Select ASCII („Z") ---- 90

CHAR (): A string function that converts an **int** ASCII code to a character.

Ex: Select CHAR (90)----- Z

CHARINDEX (): Returns the starting position of the specified expression in a character string.

Ex: Select CHARINDEX („S",'SUDHAKAR") --------1

LEFT (): Returns the left part of a character string with the specified number of characters.

Ex: Select LEFT („SUDHAKAR", 5) ---- SUDHA

RIGHT (): Returns the right part of a character string with the specified number of characters.

Ex: Select RIGHT („SUDHAKAR", 3) ------ KAR

LEN (): Returns the number of characters, rather than the number of bytes, of the given string expression.

Ex: Select LEN („WELCOME") ---------- 7

LOWER (): Returns a character expression after converting uppercase character data to lowercase.

Ex: Select LOWER („SAI") --------sai

UPPER (): Returns a character expression with lowercase character data converted to uppercase.

Ex: Select UPPER („sai") ------ SAI

LTRIM (): Returns a character expression after removing leading blanks.

Ex: Select LTRIM („ HELLO") ------- HELLO

RTRIM (): Returns a character string after truncating all trailing blanks.

Ex: Select RTRIM („HELLO „) ------ HELLO

REPLACE (): Replaces all occurrences of the second given string expression in the first string expression with a third expression.

Ex: Select REPLACE („JACK AND JUE", „J", „BL") ------- BLACK AND BLUE

REPLICATE (): Repeats a character expression for a specified number of times.

Ex: Select REPLICATE („SAI", 3) ------- SAISAISAI

REVERSE (): Returns the reverse of a character expression.

Ex: Select REVERSE („HELLO") -------- OLLEH

SPACE (): Returns a string of repeated spaces.

Ex: Select („SAI"+SPACE (50) +"SUDHAKAR") -----SAI SUDHAKAR

SUBSTRING (expression, start, length): Returns a part of a string from expression from starting position, where length is no. of chars to be picked.

Ex: Select SUBSTRING („HELLO", 1, 3) ----------HEL

Select SUBSTRING („HELLO", 3, 3) ---------- LLO

Date and Time Functions:

These functions perform an operation on a date and time input value and return a string, numeric, or date and time value.

GETDATE (): Returns the current system date and time in the SQL Server standard internal format for date time values.

Ex: Select GETDATE () ------- 2014 02-15 15:35:22.670

DAY (): Returns an integer representing the day date part of the specified date.

Ex: Select DAY (get date ())

MONTH (): Returns an integer that represents the month part of a specified date.

Ex: Select MONTH (get date ())

YEAR (): Returns an integer that represents the year part of a specified date.

Ex: Select YEAR (get Date ())

GETUTCDATE (): Returns the date time value representing the current UTC time (Coordinated Universal Time).

Ex: Select GETUTCDATE ();

DATE NAME (): Returns a character string representing the specified date part of the specified date.

Ex: Select DATE NAME (DW, get date ())

DATE PART (): Returns an integer representing the specified date part of the specified date.

Ex: Select DATEPART (DD, get date ())

DATE ADD (): Returns a new date time value based on adding an interval to the specified date.

Ex: Select DATEADD (DD, 5, get date ())

DATE DIFF (): Returns the difference between the start and end dates in the give date part format.

Ex: Select DATEDIFF (MM, „2012-12-15", get date ())

Conversion Functions:

These functions are used to convert one data type to another. We have two conversion functions are CAST and CONVERT both provide similar functionality.

CAST (): Convert to one data type to another type.

Syntax: CAST (Expression as data type [size])

Ex: Select CAST (10.2587 as Int) ---------- 10

CONVERT (): Convert function can be used to display date time data in different format.

Syntax: Convert (Data type [size], Expression, Style value)

Ex: Select Convert (Varchar (24), get date (), 113)

The table below represents the style values for date time or small date time conversion to character data:

Sno	Value	Output	Standard
-	0 or 100	mon dd yyyy hh:mi AM (or PM)	Default
1	101	mm/dd/yy	USA
2	102	yy.mm.dd	ANSI
3	103	dd/mm/yy	British/French
4	104	dd.mm.yy	German
5	105	dd-mm-yy	Italian
6	106	dd mon yy	
7	107	Mon dd, yy	
8	108	hh:mm:ss	
-	9 or 109	mon dd yyyy hh:mi:ss:mmmAM (or PM)	Default+millisec
10	110	mm-dd-yy	USA
11	111	yy/mm/dd	Japan
12	112	Yymmdd	ISO
-	13 or 113	dd mon yyyy hh:mi:ss:mmm (24h)	
14	114	hh:mi:ss:mmm (24h)	
-	20 or 120	yyyy-mm-dd hh:mi:ss (24h)	
-	21 or 121	yyyy-mm-dd hh:mi:ss.mmm (24h)	
-	126	yyyy-mm-ddThh:mi:ss.mmm (no spaces)	ISO8601
-	130	dd mon yyyy hh:mi:ss:mmmAM	Hijiri
-	131	dd/mm/yy hh:mi:ss:mmmAM	Hijiri

Aggregate functions/Group functions: Aggregate functions perform a calculation on a set of values and return a single value. Aggregate functions are often used with the GROUP BY clause of the SELECT statement.

SUM (): Returns the sum of all the values .Sum can be used with numeric columns only. Null values are ignored.

Ex: SELECT SUM (SALARY) FROM EMP

AVG (): Returns the average of the values in a group. Null values are ignored.

Ex: SELECT AVG (SALARY) FROM EMP

MAX (): Returns the maximum value in the expression.

Ex: SELECT MAX (SALARY) FROM EMP

MIN (): Returns the minimum value in the expression.

Ex: SELECT MIN (SALARY) FROM EMP

COUNT (): Returns the number of records in a table. This function again use in three ways.

1. **COUNT (*):** It Returns total number of records in a table
   Ex: SELECT COUNT (*) FROM EMP
2. **COUNT (Expression/Column name):** It returns number of records including duplicate values but not null vales.
   Ex: SELECT COUNT (ENAME) FROM EMP
3. **COUNT (Distinct Column name):** It returns number of records without null and duplicate values.
   Ex: SELECT COUNT (Distinct ENAME) FROM EMP

Distinct Key: If we use this key word on a column with in a query then it will retrieve the values of the column without duplicates.

OPERATORS IN SQL: Operator is a symbol which performs some specific operation on operands or expressions. These operators are classified into 6 types in SQL.

1. Assignment operator
2. Arithmetic operator
3. Comparison operator
4. Logical operator
5. Set operator

Assignment operator: Assignment operator contain only one operator is knows as equal „=" operator.

Ex1: Write a Query to display the employee details whose salary is equal to10000

- SELECT * FROM EMP WHERE SAL=10000

Ex2: Write a query to change the deptno as „10"whose employee id is 101

- UPDATE EMP SET DEPTNO=10 WHERE EID=101

Ex3: Write a query to delete a record whose employee id is 107

- DELETE FROM EMP WHERE EID=107

Arithmetic operator: Arithmetic operators perform mathematical operations on two expressions. The lists of arithmetic operators are + (Add), - Subtraction,* Multiplication. / (Divide) Division and % (Modulo) Returns the integer remainder of a division. For example, 12 % 5 = 2 because the remainder of 12 divided by 5 is 2.

Ex1: Select 100+250

Select 245-400

Select 20*20

Select 25/5

Select 37%6

Select 20/5+20/5

Select 35.50+20

Ex2: WAQ to find student TOTAL, AVERAGE AND CLASS OF a table

Step1: Create table student (Sid int, sname varchar (50), math"s int, phy int, che int, total int, average int, class varchar (max))

Step2: Update student set total=maths+phy+che

Step3: Update student set average=total/3

Step4: Update student set class=

Case

When average>=60 then 'First class'

When average>=50 then 'second class'

When average>=40 then 'third class'

Else

'Fail'

End

CASE (): This function is used to execute list of conditions and returns a value.

Syntax: Case

<Condition 1> ----------<Condition N>

Else

<Statement>

End

Comparison operators: Comparison operators test whether two expressions are the same. Comparison operators can be used on all expressions except expressions of the text, ntext, or image data types. The following table lists the Transact-SQL comparison operators are > (Greater Than),< (Less Than) ,>= (Greater Than or Equal To) ,<= (Less Than or Equal To) ,!= (Not Equal To),!< (Not Less Than),!> (Not Greater Than)

Examples:

- Select ename from EMP where salary<50000
- Update EMP set salary=1000 where salary>90000
- Update EMP set ename='joshitha' where salary<=25000
- Update EMP set salary=98000 where salary>=1000
- Select ename from Emp where salary !>98000
- Select ename from Emp where salary !<98000
- Select ename from Emp where salary !=98000

Logical operator: Logical operators test for the truth of some condition. Logical operators, like comparison operators, return a Boolean data type with a value of TRUE or FALSE. Logical operators are AND , OR , NOT**,** BETWEEN, NOT BETWEEN, LIKE, NOT LIKE, IN, NOT IN, EXISTS,NOT EXISTS, ANY, ALL, SOME.

Examples:

- Select * from EMP where ename='siddhu' and salary=45000
- Select * from EMP where ename='joshitha' or salary=98000
- Select * from EMP where not ename='joshitha'
- Select * from EMP where salary between 10000 and 50000
- Update EMP set ename='SAI' where eid=101 and salary=25000

Queries Using 'Select' with 'where' clause:

- Write a Query to display the employee details whose salary is less than 10000
- SELECT * FROM EMP WHERE SAL<10000

- Write a Query to display the employee details whose salary is greater than or equal to 9000 and less than 15000
- SELECT * FROM EMP WHERE SAL>=9000 AND SAL<=15000

(OR)

- SELECT * FROM EMP WHERE SAL BETWEEN 9000 AND 15000
- Write a Query to display the employee details whose salary is not between 9000 and 15000
- SELECT * FROM EMP WHERE SAL NOT BETWEEN 9000 AND15000

- Write a Query to display the employee details whose name starts with „r"
- SELECT * FROM EMP WHERE ENAME LIKE „r%"

- Write a Query to display the employee details whose name ends with „y"
- SELECT * FROM EMP WHERE ENAME LIKE „%Y"

- Write a Query to display the employee details whose name contains the letter „a"
- SELECT * FROM EMP WHERE ENAME LIKE „%A%"

- Write a Query to display the employee details whose names contains only three letters
- SELECT * FROM EMP WHERE ENAME LIKE „---„

- Write a Query to display the employee details whose names contain „r" and salary greater than 9000
- SELECT * FROM EMP WHERE ENAME LIKE „%R%" AND SAL>9000

- Write a Query to display the employee details whose greater than ram
- SELECT * FROM EMP WHERE ENAME>"RAM"
- Write a Query to display the employee details whose employee id starts with 1 and ends with 1
- SELECT * FROM EMP WHERE EID LIKE „1%1"

(SQL commands are not case sensitive and also data available in SQL also not case sensitive, in oracle Data available is case sensitive)

Queries using 'Update' with 'where' clause:

- Write a query to change the deptno as „10" whose employee id is 101, 103, 107
- UPDATE EMPSET DEPTNO=10 WHERE EID=101 OR EID=103 OR EID=107

- Write a query to change the deptno as 20 who does not have deptno
- UPDATE EMPSET DEPTNO=20 WHERE DEPTNO IS NULL

- Write a query to change the employee salaries as 12000 who are working under 10 dept and their names starts with „r"
- UPDATE EMPSET SAL=12000 WHERE DEPTNO=10 AND ENAME LIKE „R%"

- Write a query to change the deptno as 30 whose second letter is „a"
- UPDATE EMPSET DEPTNO=30 WHERE ENAME="-A%"

- Write a query to change the employee salaries as 8500 who are working under 10 and 20 deptno
- UPDATE EMPSET SAL=8500 WHERE DEPTNO=10 OR DEPTNO=20

(OR)

- UPDATE EMPSET SAL=8500 WHERE DEPTNO IN(10,20)

- Write a query to change the employee salaries as 8500 who are not working under 10 and 20 deptno
- UPDATE EMPSET SAL=8500 WHERE DEPTNO NOT IN (10,20)

- Write a query to change the employee salaries as 15000 and names ends with „m" & working under 10 deptno
- UPDATE EMPSET SAL=15000 WHERE ENAME="%M" AND DEPTNO=10

- Write a query to change the employee salaries as 5500 whose employee id ends with 4 and deptno starts with 2

- UPDATE EMPSET SAL=5500 WHERE EID LIKE „%4" AND DEPTNO LIKE „2%"

- Write a query to change the employee salaries as 25000 whose salary less than 10000 and the name contains letter „a" and working under dept 20
- UPDATE EMPSET SAL=25000 WHERE SAL<10000 AND ENAME LIKE „%A%" AND DEPTNO IN (20)

- Write a query to change the employee salaries as 10000 whose salary is greater than or equal to 8500 and less than or equal to 9000
- UPDATE EMPSET SAL=10000 WHERE SAL BETWEEN 8500 AND 9000

Set Operators: Set operators combine results from two or more queries into a single result set. SQL Server provides the following set operators.

- UNION
- UNION ALL
- INTERSECT
- EXCEPT

To combine the results of two queries we need to follow the below basic rules.

- The number and the order of the columns must be the same in all queries.
- The data types must be compatible(Well-Matched)

Example:

CREATE TABLE EMP_HYD (EID INT, ENAME VARCHAR (50), SALARY MONEY)

CREATE TABLE EMP_CHENNAI (EID INT, ENAME VARCHAR (50))

EMP_HYD

EID	ENAME	SALARY
101	SAI	25000.00
102	SIDDHU	32000.00
103	KAMAL	42000.00
104	NEETHU	63000.00

EMP_CHENNAI

EID	ENAME
101	SAI
105	POOJA
106	JASMIN

UNION: it combines the result of two or more select statements into a single result set that includes all the records belongs to all queries except duplicate values.

Ex: Select Ename from EMP_HYD

Union

Select Ename from EMP_CHENNAI

OUTPUT: ENAME

JASMIN

KAMAL

NEETHU

POOJA

SAI

SIDDHU

UNION ALL: it is same as union but returns duplicate values

Ex: Select Ename from EMP_HYD

Union ALL

Select Ename from EMP_CHENNAI

OUTPUT: ENAME

SAI

SIDDHU

KAMAL

NEETHU

SAI

POOJA

JASMIN

INTERSECT: INTERSECT returns any distinct values that are common in left and right tables.

Ex: Select Ename from EMP_HYD

Intersect

Select Ename from EMP_CHENNAI

OUTPUT: ENAME

SAI

EXCEPT: EXCEPT returns any distinct values from the left query that are not found on the right query.

Ex: Select Ename from EMP_HYD

Except

Select Ename from EMP_CHENNAI

OUTPUT: ENAME

KAMAL

NEETHU

SIDDHU

Ex: Select Ename from EMP_CHENNAI

Except

Select Ename from EMP_HYD

OUTPUT: ENAME

JASMIN

POOJA

CLAUSES IN SQL:

We can add these to a query for adding additional options like filtering the records, sorting records and grouping the records with in a table. These clauses contains the following clauses are,

WHERE: This clause is used for filter or restricts the records from the table.

Ex: SELECT * FROM EMP WHERE SAL=10000

ORDER BY: The order by clause is used to sort or arrange the data in ascending or descending order with in table. By default order by clause arrange or sort the data in ascending order only.

- If we want to arrange the records in a descending order then we use Desc keyword.
- We can apply order by clause on integer and string columns.

Ex: SELECT * FROM EMP ORDER BY EID (For Ascending Order)

Ex: SELECT * FROM EMP ORDER BY ENAME DESC (For Descending Order)

TOP N CLAUSE: This clause is used to fetch a top n number of records from a table.

Ex: SELECT TOP 3 * FROM EMP

Ex: UPDATE TOP 3 EMP SET ENAME="SAI"

Ex: DELETE TOP 3 FROM EMP

GROUP BY: Group by clause will use for to arrange similar data into groups. when we apply group by clause in the query then we use group functions like count(),sum(),max(),min(),avg().

If we use group by clause in the query, first the data in the table will be divided into different groups based on the columns and then execute the group function on each group to get the result.

Ex1: WAQ to find out the number of employees working in the organization

Sol: SELECT COUNT (*) FROM EMP

Ex2: WAQ to find out the number of employees working in each group in the organization.

Sol: SELECT DEPT, COUNT=COUNT (*) FROM EMP GROUP BY DEPT

Ex3: WAQ to find out the total salary of each department in the organization

Sol: SELECT DEPT, TOTALSALARY=SUM (SALARY) FROM EMP GROUP BY DEPT (Like this we can find max, min, avg salary in the organization)

HAVING CLAUSE: Having clause is also used for filtering and restricting the records in a table just like where clause.

Ex: WAQ to find out the number of employees in each department only if the count is greater than 3

Sol: SELECT DEPT, COUNT=COUNT (*) FROM EMP GROUP BY DEPT HAVING COUNT (*) >3

Differences Between WHERE and HAVING Clause:

WHERE	HAVING
WHERE clause is used to filter and restrict the records before grouping	HAVING clause is used to filter and restrict the records after grouping
If restriction column associated with A aggregative function then we cannot use WHERE clause there	But we can use HAVING clause at this situations
WHERE clause can apply without group by clause	HAVING clause cannot be applied without a group by clause
WHERE clause can be used for restricting individual rows	Where as HAVING clause is used along with group by clause to filter or restrict groups

SYNONYM: synonym is database object which can be created as an "alias" for any object like table, view, procedure etc.

- If we apply any DML operations on synonym the same operations automatically effected to corresponding base table and vice versa.
- If we create a synonym, the synonym will be created on entire table. It is not possible to create the synonym on partial table.
- When we create synonym based on another the new synonym does not allow us to perform any DML operations because Synonym chaining is not allowed.
- Synonym will become invalid into two cases,
    1. When we drop the base table
    2. When we change the base table name

- On invalid synonym we cannot apply any DML operations and we cannot create synonym based on more than one table at a time.
- When we change the structure of the base table the corresponding synonym automatically reflected with same changes.
- But, if we change the structure of the synonym that is not reflected to the base table because we cannot change the structure of the synonym.

Syntax: Create synonym <synonym name> for <object name>

Ex: Create synonym synemp for employee

Syntax to drop a synonym: Drop synonym <synonym name>

Ex: Drop synonym synemp

Syntax to Creating a table from an existing table:

we can create a table from an existing table and maintain a copy of the actual table before manipulating the table.

Syntax: Select * into <New Table Name> from <Old Table Name>

Ex1: Select * into New_Emp from Employee

In this case it creates a table New_Emp by copying all the rows and columns of the Employee table.

Ex2: Select EID, ENAME into Test_Emp from Employee

In this case it creates a table Test_Emp with only the specified columns from the employee table.

Ex3: Select * into Dummy_Emp from employee where 1=2

In this case it creates the Dummy table without any data in it.

Copying data from one existing table to another table:

We can copy the data from one table to another table by using a combination of insert and select statement as following

Syntax: Insert into <Dummy Table name> select * from <Table Name>

Ex: Insert into Dummy_Emp select * from Employee

Constraint in SQL

Why Constraint in SQL: Constraint is using to restrict the insertion of unwanted data in any columns. We can create constraints on single or multiple columns of any table. It maintains the data integrity i.e. accurate data or original data of the table. Data integrity rules fall into three categories:

- Entity integrity
- Referential integrity
- Domain integrity

Entity Integrity: Entity integrity ensures each row in a table is a uniquely identifiable entity. You can apply entity integrity to a table by specifying a PRIMARY KEY and UNIQUE KEY constraint.

Referential Integrity: Referential integrity ensures the relationships between tables remain preserved as data is inserted, deleted, and modified. You can apply referential integrity using a FOREIGN KEY constraint.

Domain Integrity: Domain integrity ensures the data values inside a database follow defined rules for values, range, and format. A database can enforce these rules using CHECK KEY constraints.

Types of constraints in SQL Server:-

1. Unique Key constraint.
2. Not Null constraint.
3. Check constraint
4. Primary key constraint.
5. Foreign Key constraint.

1. Unique Key:- Unique key constraint is use to make sure that there is no duplicate value in that column. Both unique key and primary key both enforces the uniqueness of column but there is one difference between them unique key constraint allow null value but primary key does not allow null value.

In a table we create one primary key but we can create more than one unique key in Sql Server.

Ex: `create table EMP(EID int unique,ENAME varchar(50) unique,SALARY money);`

2. Not null constraint: - Not null constraint is used to restrict the insertion of null value at that column but allow duplicate values.

Ex: `create table EMP(EID int not null,ENAME varchar(50) not null,SALARY money);`

3. Check Constraint: - This constraint is using to check value at the time of insertion like as salary of any employee is always greater than zero. So we can create a check constraint on employee table which is greater than zero.

Ex: `create table emp4(eno int,ename varchar(50),age int check (age between 20 and 30))`

4. Primary Key:- Primary key is a combination of unique and not null which does not allow duplicate as well as null values into a column. In a table we create one primary key only.

Ex:
```
create table emp(EID int primary key,ENAME varchar(50),SALARY money)
```

5. Foreign Key: - One of the most important concepts in database is creating relationships between database tables. These relationships provide a mechanism for linking data stored in multiple tables and retrieving it in an efficient manner.

In order to create a link between two tables we must specify a foreign key in one table that references a column in another table**.**

Foreign key constraint is used for relating or binding two tables with each other and then verifies the existence of one table data in the other.

To impose a foreign key constraint we require the following things.

We require two tables for binding with each other and those two tables must have a common column for linking the tables.

Example:

To create Department Table (PARENT TABLE):-

create table **Department**(Deptno int primary key,DNAME varchar(50),LOCATION varchar(max))

Insert Records Into Department Table:

```
insert into Department values(10,'Sales','Chennai')
insert into Department values(20,'Production','Mumbai')
insert into Department values(30,'Finance','Delhi')
insert into Department values(40,'Research','Hyderabad')
```

To create Employee Table(CHILD TABLE):-

create table **Employee**(EID int,ENAME varchar(50),SALARY money,Deptno int foreign key references **Department**(Deptno))

JOINS IN SQL: Joins are used for retrieving the data from more than one table at a time. Joins can be classified into the following types.

- EQUI JOIN
- INNER JOIN
- OUTER JOIN
- LEFT OUTER JOIN

- RIGHT OUTER JOIN
- FULL OUTER JOIN
- NON EQUI JOIN
- SELF JOIN
- CROSS JOIN
- NATURAL JOIN

EQUI JOIN: If two or more tables are combined using equality condition then we call as an Equi join.

Ex: WAQ to get the matching records from EMP and DEPT tables

Sol: SELECT * FROM EMP, DEPT WHERE (EMP.EID=DEPT.DNO) (NON-ANSI STANDARD)

Sol: SELECT E.EID, E.ENAME, E.SALARY, D.DNO, D.DNAME FROM EMP E, DEPT D WHERE E.EID=D.DNO (NON-ANSI STANDARD)

INNER JOIN: Inner join return only those records that match in both table

Ex: SELECT * FROM EMP E INNER JOIN DEPT D ON E.EID=D.DNO (ANSI)

OUTTER JOIN: It is an extension for the Equi join. In Equi join condition we will be getting the matching data from the tables only. So we loss UN matching data from the tables.

To overcome the above problem we use outer join which are used to getting matching data as well as UN matching data from the tables. This outer join again classified into three types

LEFT OUTER JOIN: It will retrieve or get matching data from both table as well as UN matching data from left hand side table

Ex: SELECT * FROM EMP LEFT OUTER JOIN DEPT ON EMP.EID=DEPT.DNO;

RIGHT OUTER JOIN: It will retrieve or get matching data from both table as well as UN matching data from right hand side table

Ex: SELECT * FROM EMP RIGHT OUTER JOIN DEPT ON EMP.EID=DEPT.DNO;

FULL OUTER JOIN: It will retrieve or get matching data from both table as well as UN matching data from left hand side table plus right hand side table also.

Ex: SELECT * FROM EMP FULL OUTER JOIN DEPT ON EMP.EID=DEPT.DNO;

NON EQUI JOIN: If we join tables with any condition other than equality condition then we call as a non Equi join.

Ex: SELECT * FROM EMP, SALGRADE WHERE (SALARY > LOWSAL) AND (SALARY < HIGHSAL)

SELF JOIN: Joining a table by itself is known as self join. When we have some relation between the columns within the same table then we use self join.

When we implement self join we should use alias names for a table and a table contains any no. of alias names.

Ex: SELECT E.EID, E.ENAME, E.SALARY, M.MID, M.ENAME MANAGERSNAME, M.SALARY FROM EMP E, EMP M WHERE E.EID=M.MID.

CROSS JOIN: Cross join is used to join more than two tables without any condition we call as a cross join. In cross join each row of the first table join with each row of the second table.

So, if the first table contain „m" rows and second table contain „n" rows then output will be „m*n" rows.

Ex: SELECT * FROM EMP, DEPT

Ex: SELECT * FROM EMP CROSS JOIN DEPT

NATURAL JOIN: It is used to avoid duplicate column from the tables.

EX: SELECT EID, ENAME, SALARY, DNO, DNAME FROM EMP E, DEPT D WHERE E.EID=D.DNO

Making To Joins Three Tables:

CREATE TABLE STUDENTS (SID Int, SNAME varchar (20), SMBNO char (10), CID Int)

INSERT STUDENT VALUES (1,'aa','7894561233', 10), (2,'bb','9874563211', 20), (3,'cc','8749653215', 30), (4,'dd','7788996655', 40)

CREATE TABLE COURSES (CID int, CNAME Varchar (20), CFEE decimal (6, 2))

INSERT COURSES VALUES (10,'c', 500), (20,'c++', 1000), (50,'sql', 1800), (60,'.net', 3500), (70,'sap', 8000)

CREATE TABLE REGISTER (SNO int, REGDATE date, CID Int)

INSERT REGISTER VALUES (100,'2014/10/20', 10), (101,'2014/10/21', 80), (102,'2014/10/22', 90)

Select * from course c inner join student s on c.cid=s.cid inner join register r on s.cid=r.cid

Select * from student s left outer join course c on s.cid=c.cid left outer join register r on c.cid=r.cid